

MCP-Backed Chat: Angular Frontend Architecture Plan

Routing browser-submitted queries to Claude Code (with existing MCP tool servers) | Prepared 2026-08-13

1. Objective

Today, queries are sent to Claude Code from a terminal, and Claude Code reaches two configured MCP servers to gather tool results and formulate answers. The objective is to let end users submit the same kind of queries from a web browser, via an Angular frontend, without losing MCP tool access or multi-turn context.

2. Recommended Approach

Do not connect the browser directly to Claude or to the MCP servers. Introduce a thin **backend bridge service** between the Angular app and Claude. The browser talks to your bridge over HTTP/WebSocket; the bridge is the only component that holds API credentials and talks to Claude and the MCP servers.

Key decision: drive Claude programmatically from the bridge using the **Claude Agent SDK** — `@anthropic-ai/claude-agent-sdk` on npm (Python equivalent: `claude-agent-sdk`) — rather than shelling out to the `claude` terminal binary as a subprocess. The SDK exposes a `query()` call that returns an async generator of typed `SDKMessage` events (text deltas, tool-call start/result, final result with session id), and it auto-discovers the project's existing `.mcp.json`, so the two MCP servers already configured for the CLI work with no changes. A CLI-subprocess wrapper (`claude -p "..."` `--output-format stream-json --include-partial-messages --resume <id>`) is a valid fallback if you'd rather keep using the CLI unchanged, but is more fragile in a long-running service (process lifecycle, stdout parsing, TTY quirks).

Why a bridge service instead of calling Claude straight from Angular

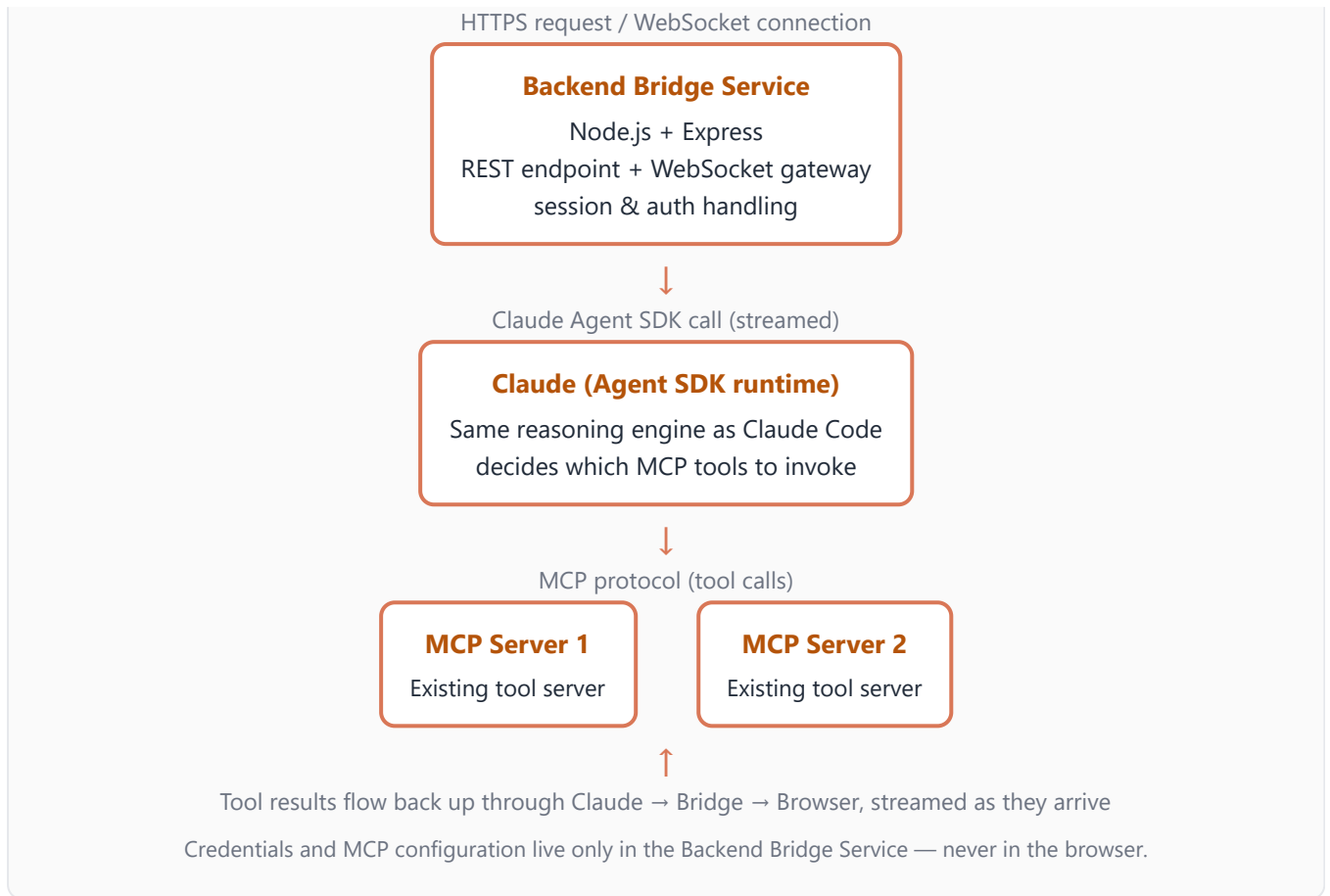
- Credentials (Anthropic API key) must never reach the browser.
- MCP servers are typically local/stdio or trusted-network processes — they should stay behind a server, not be exposed to arbitrary browser clients.
- You need a place to hold per-user session/conversation state, apply auth, rate-limit, and log tool usage.

3. System Flow Diagram

Angular Frontend

Chat UI in browser
submits user query





4. Component Breakdown

Component	Responsibility	Notes
Angular frontend	Chat UI: input box, message list, streaming render, session/history display	Use signals for message state; render markdown/code blocks; show a distinct indicator when a tool call is in progress
Backend bridge (Node.js/TypeScript)	Authenticates users, opens a Claude Agent SDK session per conversation, streams events to the client, persists minimal session metadata	TypeScript keeps types shared with Angular; Express or Fastify for REST, native <code>ws</code> or Socket.IO for streaming
Claude Agent SDK	Runs the same query/response/tool-use loop Claude Code runs in the terminal	Configured with the same <code>mcpServers</code> definitions already used by the CLI
MCP Server 1 & 2	Expose existing tools (unchanged)	No changes required if they're reachable from wherever the bridge process runs

5. Step-by-Step Implementation Plan

1. **Confirm current MCP configuration.** Locate the existing `.mcp.json` (or `~/.claude.json`) that declares the two MCP servers today — `command`, `args`, `env`, `transport` — so it can be reused as-is.
2. **Stand up the backend bridge service.** New Node.js/TypeScript project (Express or Fastify). Install `@anthropic-ai/claude-agent-sdk` and set `ANTHROPIC_API_KEY` as a server-side environment variable (never shipped to the browser). Run in "bare"/API-key mode so no interactive OAuth/keychain login is required.
3. **Wire up MCP access from the SDK.** Either rely on auto-discovery of the project's `.mcp.json`, or pass the same server definitions explicitly via the `mcpServers` option on `query()`, so the bridge has identical tool access to the terminal session.
4. **Build a streaming query endpoint.** Expose a WebSocket (preferred) or Server-Sent-Events endpoint that accepts a user message, calls `query()`, and forwards each `SDKMessage` (text delta, tool-call start/result, final result) to the client as it arrives.
5. **Add session/conversation continuity.** Capture the `session_id` from each result, store it per browser conversation, and pass it back via the `resume` option on the next `query()` call so multi-turn context survives across separate HTTP/WebSocket calls.
6. **Add auth and guardrails.** Require an authenticated user/session before accepting queries; add rate limiting and input validation; keep MCP servers and the API key unreachable from the client.
7. **Build the Angular chat UI.** Message list + input component, a service that opens the WebSocket/SSE connection and exposes streamed tokens as signals, rendering of tool-call status ("using tool: ...") distinct from final answer text.
8. **Wire frontend to backend end-to-end.** Connect the Angular service to the bridge's streaming endpoint; verify a full round trip: browser query → bridge → Claude → MCP tool call → streamed answer back to the browser.
9. **Test the golden path and edge cases.** Long-running tool calls, dropped WebSocket connections/reconnect, errors surfaced from an MCP server, empty/invalid queries, concurrent sessions from multiple browser tabs.
10. **Deploy.** Containerize the bridge service; serve the Angular production build (static hosting or same Node process); put both behind a reverse proxy that supports WebSocket upgrade (e.g. Nginx); configure CORS to only allow the frontend origin.

6. Key Technical Considerations

- **Streaming shape:** design the WebSocket message protocol up front — e.g. distinct event types for `text_delta`, `tool_call_started`, `tool_call_result`, `done`, `error` — so the Angular UI can render intermediate tool activity, not just a final blob of text.
- **Session resumption:** each browser message may hit the bridge as an independent request; the bridge must resume the correct underlying Claude session (by session id) rather than starting a fresh context every turn.
- **Concurrency:** the bridge should support multiple simultaneous user sessions without cross-talk — keep session state keyed per connection/user, not in module-level globals.

- **Security boundary:** the browser should never receive the Anthropic API key, raw MCP server connection details, or unfiltered tool output that might contain sensitive data.
- **Fallback option:** if the Agent SDK doesn't cover a needed capability, the same architecture works with the bridge spawning the `claude` CLI as a subprocess per query instead — only the "Claude (Agent SDK runtime)" box in the diagram changes.

7. Quick Reference

Item	Value
TypeScript SDK package	<code>@anthropic-ai/claude-agent-sdk</code>
Python SDK package (alt.)	<code>claude-agent-sdk</code>
Existing MCP config file	<code>.mcp.json</code> (project) or <code>~/.claude.json</code> (user), auto-discovered by the SDK
Credential	<code>ANTHROPIC_API_KEY</code> environment variable on the backend only
Streaming (CLI fallback)	<code>claude -p "... " --output-format stream-json --include-partial-messages -resume <id></code>
Session continuity	<code>resume: session_id</code> option on <code>query()</code> , captured from the prior result event

8. Open Questions to Resolve Before Building

- Do users need individual authenticated accounts, or is this behind an already-trusted network/VPN?
- Should conversation history persist across browser sessions (requiring storage), or is in-memory-per-connection acceptable initially?
- Are the two MCP servers reachable from wherever the bridge service will be hosted (network/firewall considerations if not co-located)?
- Expected concurrency: single user, small internal team, or public-facing?

Plan generated to guide implementation of an Angular frontend that routes browser queries to Claude Code / the Claude Agent SDK, preserving existing MCP tool access.